

Compilers

Project 7: Compilation to MIPS and binary

2nd Bachelor Computer Science
2025 – 2026

Kasper Engelen
`kasper.engelen@uantwerpen.be`

For the project of the Compilers course you will develop, in groups of 3 students, a compiler capable of translating a program written in (a subset of) the C language into the LLVM intermediate language, which is in turn compiled into MIPS and binary code. The project will be completed over the semester with weekly incremental assignments. It is recommended to complete **one assignment per week!**

Concretely, you will construct a grammar and lexer specification, which is then turned into Python code using the ANTLR tool. The rest of the project will consist of developing custom code written in Python 3. This custom code will take the parse tree and turn it into an abstract syntax tree (AST). This AST will then be used to generate code in the LLVM IR language, which is in turn compiled into MIPS and binary code. At various stages of the project you will also have to develop utilities to provide insight into the internals of the compiler, such as visualising the AST tree.

On Blackboard you can find two appendices: one in which an overview of the project is given, and one which contains some information on how to use ANTLR.

The goal of this assignment is to extend your compiler to support compilation to native binary code, as well as MIPS assembly code. An essential part of this assignment is figuring out how to use existing tooling to do this (command-line tools or libraries only). You are expected to achieve this by going through the documentation of existing tools such as LLVM, Clang, GCC, etc.

1 Compilation to MIPS and binary

1.1 Functionality

- (mandatory) Extend your compiler to output native binary code (e.g., x86, ARM). You may make use of existing tooling. Do not use GUI-based tools and do not use online tools. Chain all tools together such that they work together automatically. “Manual” setups are not accepted!
- (mandatory) Extend your compiler to output MIPS code. You **MUST** use existing tooling. Do not use GUI-based tools and do not use online tools. Do not manually construct the MIPS code.
- (mandatory) Demonstrate that the MIPS code can work in either SPIM or MARS.
- (mandatory) Refer to the documentation of the tooling (e.g., LLVM documentation) to motivate your design choices. ChatGPT will not be accepted here.
- (mandatory) You need to re-use your LLVM IR output for this. Do not write an x86 or MIPS backend from scratch.

1.2 Error checks

All error checks from the previous parts should still work.